Cloud Computing (CS3081): Mini Project Technical Report

Knowledge Distillation and Deployment of a Lightweight Command-Parsing Language Model on a Local Cloud

Utkarsh Singh | Roll Number: 230101079
Department of Computer Science & Engineering

GitHub Repository: https://github.com/Unomison/Cloud-computing-assignment-knowledge-distillation-
November 2026

Abstract

This technical report documents the end-to-end design, implementation, evaluation, and cloud deployment of a sequence-level knowledge-distilled command-parsing language model. A teacher model (`phi3.5`, 3.8B parameters) running under Ollama was distilled into an ultra-compact student model (`EleutherAI/pythia-70m`, 70M parameters) capable of parsing natural language voice commands into an exact 4-key JSON schema. We detail parallel shard labelling, schema cleaning, CPU-based distillation training, lightweight containerization (≤ 2 GB), local cloud orchestration on k3d/k3s Kubernetes with SeaweedFS object storage, automated HPA autoscaling, rolling updates, stateless IPv4 subnetting and NACL security, and a 5-stage automated CI/CD quality gate with Gitleaks security auditing.

1 Task 1: Object Storage and Parallel Teacher Labelling

1.1 Architecture and Storage Setup

A multi-namespace local Kubernetes cluster was deployed on k3d using the declarative specification in `cluster/k3d.yaml`. For scalable S3-compatible object storage, SeaweedFS was deployed in the `data` namespace with a dedicated persistent volume claim and an internal ClusterIP service (`seaweedfs-s3.data.svc.cluster.local:8333`). Two distinct object storage buckets were initialized:

- `datasets`: Hosts raw command corpora (`commands_pool.jsonl`), distributed teacher shard outputs (`datasets/teacher/`), and the cleaned training dataset (`datasets/v1/train.jsonl`).
- `models`: Stores trained student model artifacts, tokenizers, and SHA-256 verification manifests across deployment versions (`models/v1/` and `models/v2/`).

1.2 Teacher Prompt and Sharded Parallel Labelling

The teacher model (`phi3.5`, Phi-3.5-mini-instruct, 3.8B parameters) was queried through Ollama with fixed deterministic parameters: `temperature = 0.0` and `seed = 42`. The prompt in `prompts/teacher_prompt.txt` enforces zero-shot extraction into a strict 4-key JSON format (`intent`, `date`, `time`, `place`) adhering to the eight SLURP intents.

To distribute computational workload across CPU cores, parallel worker pods were deployed in the `pipeline` namespace. Sharding is governed by modular hashing:

$$\text{Worker Assignment} = \{i \in \mathbb{N} \mid i \pmod{\text{SHARD_COUNT}} = \text{SHARD_INDEX}\} \tag{1}$$

With SHARD_COUNT $= 2$, Worker 0 processed even command line indices ($i = 0, 2, 4, \dots$) generating `shard-0.jsonl` (1,500 records), while Worker 1 processed odd indices ($i = 1, 3, 5, \dots$) generating `shard-1.jsonl` (1,500 records). Every command ID appears in exactly one shard with zero gaps and zero repeats.

1.3 Data Cleaning and Verification

The cleaning utility `worker/clean.py` aggregates all shards and enforces mutually exclusive filtering across five failure modes, prioritizing the smallest ID for duplicates:

(a) **Invalid JSON**: Syntax decoding failure or truncated output.

(b) **Key Mismatch**: Keys deviate from exact set $\{\text{intent}, \text{date}, \text{time}, \text{place}\}$.

(c) **Invalid Intent**: Intent not in the 8 authorized voice assistant classes.

(d) **Slot Hallucination**: Non-null slot text does not appear in command string.

(e) **Duplicate Commands**: Redundant textual commands (preserving earliest ID).

Integrity Check: Input $= (a) + (b) + (c) + (d) + (e) + \text{Kept} = 14 + 10 + 9 + 14 + 47 + 2906 = 3000$ (Verified True). The SHA-256 checksum recorded in `datasets/v1/train.sha256` is:

`cce578baad03646ee65e9a72f0c039da795b21c842840d752055e17a477c230c`

Table 1: Dataset Cleaning Breakdown (Total Input = 3,000)

Category / Reason	**Count**	**Percentage (%)**
Total Input Records	3,000	100.00%
(a) Invalid JSON	14	0.47%
(b) Incorrect Keys	10	0.33%
(c) Invalid Intent	9	0.30%
(d) Hallucinated Slot	14	0.47%
(e) Duplicate Commands	47	1.57%
Kept Records (`datasets/v1/train.jsonl`)	**2,906**	**96.87%**

2 Task 2: Training the Student (Sequence-Level Distillation)

2.1 Distillation Methodology

Sequence-level knowledge distillation (Kim & Rush, 2016) was employed to transfer task capabilities from the 3.8B Phi-3.5 teacher into the 70M-parameter `EleutherAI/pythia-70m` student model. The student is trained directly on the teacher's synthesized sequences:

$$\mathcal{L}_{\text{seq}}(\theta) = -\sum_{t=1}^{|\mathbf{y}|} \log P_\theta(y_t \mid y_{<t}, \mathbf{x}_{\text{cmd}}) \tag{2}$$

where $\mathbf{x}_{\text{cmd}}$ is formatted as `Command: <text>`, and target $\mathbf{y}$ is formatted as `JSON: <teacher_reply>`.

2.2 Hyperparameters and Reproducibility

All six mandatory hyperparameter items are codified in `training/config.yaml`:

- **Random Seed**: 42 (fixed in code via PyTorch set_seed)
- **Learning Rate**: 5.0×10^{-4} with linear warmup
- **Batch Size**: 8 (optimized for CPU cache locality)
- **Epochs**: 3 (version 1), 4 (version 2)
- **Max Sequence Length**: 128 tokens
- **Library Versions**: PyTorch `2.2.2`, Transformers `4.39.3`, Datasets `2.18.0`, Accelerate `0.28.0`.

Training is triggered reproducibly via `make train` or quick sanity-testing via `make train SUBSET=50`. Per-epoch losses recorded in `training/loss.csv` show smooth convergence: Epoch 1: 2.2583, Epoch 2: 0.8117, Epoch 3: 0.5517.

2.3 Evaluation and Comparative Findings

All models were evaluated against the 100 ground-truth examples in `data/validation.jsonl` using `score.py`:

Table 2: Validation Benchmark Comparison Across Models (in %)

Evaluation Metric	**Teacher (Phi-3.5)**	**Untrained Pythia-70M**	**Distilled Student (v1)**
JSON Validity ($\geq$ 95% gate)	100.0%	0.0%	**99.0%**
Intent Accuracy	100.0%	0.0%	**99.0%**
Date Accuracy	100.0%	0.0%	**96.0%**
Time Accuracy	100.0%	0.0%	**99.0%**
Place Accuracy	100.0%	0.0%	**99.0%**
Full-Frame Accuracy	**100.0%**	**0.0%**	**96.0%**

2.4 Distillation Theory: Sequence-Level vs. Logit-Based Distillation

1. **Sequence-Level Distillation**: The student is trained on the discrete token sequences sampled or greedily decoded from the teacher model. The objective optimizes standard cross-entropy loss over the teacher's chosen sequence, requiring access only to generated output text.

2. **Logit-Based Distillation**: The student optimizes Kullback-Leibler (KL) divergence over the teacher's full probability distribution across the entire vocabulary at every generation step:

$$\mathcal{L}_{\text{logit}} = \alpha \mathcal{L}_{\text{CE}} + (1-\alpha) T^2 \mathcal{D}_{\text{KL}} \left(\sigma \left(\frac{\mathbf{z}_{\text{teacher}}}{T} \right) \parallel \sigma \left(\frac{\mathbf{z}_{\text{student}}}{T} \right) \right) \tag{3}$$

3. **Why Logit-Based Distillation Is Impossible Here**: Logit distillation requires identical vocabulary tokenization schemes between teacher and student. Phi-3.5 uses a 32,064-token SentencePiece/Llama-based BPE tokenizer, whereas Pythia-70M uses a 50,304-token GPT-NeoX byte-level BPE tokenizer. Because the logit dimensions and token ID boundaries do not match ($|\mathcal{V}_T| \neq |\mathcal{V}_S|$), mathematical distribution matching cannot be computed. Furthermore, Ollama exposes black-box text generation endpoints without exposing soft probability logits.
4. **Limitation**: Sequence distillation discards the dark knowledge in unselected token distributions (e.g., token confidence, semantic ambiguities), making the student prone to compounding error cascades when generating open-ended sequences.

3 Task 3: Inference Service in a Container

3.1 Microservice Architecture

The web inference service is implemented in `service/app.py` listening on port 8000. It adheres strictly to the four-variable specification:

- `MODEL_URI`: Points to object storage artifact path (`models/v1`).
- `MODEL_VERSION`: Version identifier string (`v1`).
- `S3_ENDPOINT`: SeaweedFS S3 endpoint URL.
- `MAX_NEW_TOKENS`: Maximum generated sequence length (64).
- `FAULT_INJECT`: Diagnostic fault trigger (1 = replace output with 'not json').

3.2 Endpoints and Probe Implementation

- `POST /parse`: Receives {`"text": "..."`}, performs inference, validates against `schema.json`, and returns HTTP 200 with the 4-key JSON body along with headers `X-Model-Version` and `X-Served-By` (hostname/pod ID).
- `GET /healthz`: Returns HTTP 200 continuously as a liveness check while the process is alive.
- `GET /readyz`: Returns HTTP 503 while the background model download is underway from SeaweedFS, and transitions to HTTP 200 immediately upon weights readiness.

3.3 Container Optimization and Memory Verification

To ensure the container remains strictly under 2 GB, `service/Dockerfile` builds on `python:3.11-slim` and fetches CPU-only PyTorch (`torch==2.2.2+cpu`), eliminating all CUDA/cuDNN binaries. The resulting image size is **1.04 GB** ($\approx 52\%$ of the limit).

- Test: 50 sequential POST requests to `/parse` under limits (1 CPU, 1 GiB RAM).
- Baseline RSS Memory: 218.4 MiB; Peak RSS Memory: **231.2 MiB** (22.5% of 1 GiB limit).
- OOM (Out-of-Memory) Kills: **0**.
- Fault Injection Test (`FAULT_INJECT=1`): Returns HTTP 500 {`"error": "invalid_model_output"`} with 100% reliability.

4 Task 4: Kubernetes Deployment, Scaling, and Reliability

4.1 Cluster Resources and Ingress

All manifests are organized under `manifests/app.yaml` deploying to the `app` namespace:

- **Deployment**: 2 initial replicas, requests (250m CPU, 512Mi RAM), limits (1000m CPU, 1Gi RAM).
- **Probes**: Liveness on `/healthz` (period 10s), Readiness on `/readyz` (period 3s, delay 2s).
- **Ingress**: Traefik Ingress controller routing `/parse` on cluster port 80 (forwarded from host 8080).

Table 3: Kubernetes Operational Resilience Verification

Operational Test	Observed Behavior	Threshold	Status
HPA Autoscaling	Scaled 1 → 2 replicas at 74% CPU; scaled down after load	Target: 70%	Passed
Rolling Update ($v1 \rightarrow v2$)	Zero-downtime surge; error rate: 0.15%	Max: 0.5%	Passed
Rollback (`rollout undo`)	Restored `X-Model-Version: v1` across all pods	100% match	Passed
Pod Recovery	Deleted active pod during 45 req/s; 8 errors (0.33%)	Max: 2.0%	Passed
Cold-Start Time	3 runs: 4.2s, 3.9s, 4.0s (Median: **4.0s**)	S3 download verified	Passed
Load Balancing Split	Pod A: 2,724 (50.5%), Pod B: 2,670 (49.5%)	$\geq 30\%$ each	Passed

4.2 Autoscaling, Failover, and Rolling Updates

Local Cluster Limitation: The k3d cluster executes as Docker containers on a single physical host laptop. While it exercises real Kubernetes API controllers, cgroups, network namespaces, and Traefik routing, it provides no hardware fault tolerance: host OS restarts, kernel panics, or machine power loss terminate the entire cluster simultaneously.

5 Task 5: Subnetting and Network Access Control

5.1 Subnet Allocation for Block `10.20.0.0/16`

Subnets were allocated sequentially from largest host requirement to smallest, starting at `10.20.0.0`, using the smallest prefix satisfying Usable Hosts $= 2^{32-\text{prefix}} - 2 \geq$ Hosts Needed.

Table 4: Complete IPv4 Subnet Allocation Table

Subnet Name	Network Address	Prefix	Usable Host Range	Broadcast Address	Usable Hosts
app	`10.20.0.0`	`/22`	`10.20.0.1` – `10.20.3.254`	`10.20.3.255`	1,022
public	`10.20.4.0`	`/24`	`10.20.4.1` – `10.20.4.254`	`10.20.4.255`	254
data	`10.20.5.0`	`/25`	`10.20.5.1` – `10.20.5.126`	`10.20.5.127`	126
management	`10.20.5.128`	`/27`	`10.20.5.129` – `10.20.5.158`	`10.20.5.159`	30

5.2 NACL Rules and Verification

The public subnet NACL (`network/nacl.json`) enforces stateless ingress and egress boundaries. Checked against `network/packets_sample.jsonl` with `tools/nacl_sim.py --check`, the rule set achieved **100.0% accuracy (6/6)**:

Table 5: Public Subnet NACL Rule Table (`network/nacl.json`)

Dir	Rule	Proto	Port Range	CIDR	Action	Description
Inbound	100	ALL	Any	`198.51.100.0/24`	DENY	Block abusive subnet
Inbound	200	TCP	80	`0.0.0.0/0`	ALLOW	Public HTTP traffic
Inbound	210	TCP	443	`0.0.0.0/0`	ALLOW	Public HTTPS traffic
Inbound	300	TCP	22	`10.20.5.128/27`	ALLOW	SSH from management
Inbound	400	TCP	1024–65535	`10.20.0.0/22`	ALLOW	App server return traffic
Outbound	100	TCP	8000	`10.20.0.0/22`	ALLOW	Forward to app backend
Outbound	200	TCP	1024–65535	`0.0.0.0/0`	ALLOW	Return replies to clients

Stateless Reply Traffic: NACLs lack connection state tracking. Unlike stateful firewalls which automatically associate reply packets with active connections, a stateless NACL evaluates every packet independently. When external clients query port 80/443, responses must be explicitly permitted through outbound rule 200 to client ephemeral ports (1024–65535). Similarly, backend responses from the app subnet returning to the load balancer must be explicitly permitted by inbound rule 400.

Rule Ordering Significance: NACL rules are evaluated sequentially in ascending numerical order (*first-match-wins*). Because `198.51.100.0/24` is a subset of the global internet range `0.0.0.0/0`, rule 100 must

precede rules 200 and 210. If the deny rule were numbered > 210, malicious packets from `198.51.100.0/24` targeting web ports would match the allow rule first and bypass security.

6 Task 6: Reproducibility, Testing, and CI/CD

6.1 Automation Lifecycle

System lifecycle is unified in the `Makefile`:

- `make up`: Creates k3d cluster, applies SeaweedFS storage, and deploys the inference app.
- `make down`: Completely tears down cluster and resources.
- `make test`: Runs unittests, model quality scoring, and NACL simulation.

6.2 Continuous Integration Pipeline

The GitHub Actions workflow (`.github/workflows/ci.yml`) executes 5 sequential stages:

1. **Linting**: Flake8 checks syntax, unassigned variables, and formatting.
2. **Unit Tests**: Runs 4 automated test suites covering valid schema validation, rejected keys/intents, command extraction, and NACL packet decisions.
3. **Model Evaluation**: Fetches student model release assets and computes validation scores.
4. **Quality Gate**: Enforces validation via `score.py --min-validity 95`. Deliberately running an untrained base model fails with exit code 1 (`json_validity 0.0 < 95.0`), confirming gate efficacy (Workflow run link documented in README).
5. **Secret Scanning**: Gitleaks scans commit history and reports 0 leaks.

7 Task 7: Performance, Cost, and Review

7.1 Locust Load-Test Results

Under a 2-minute Locust benchmark with 4 concurrent users spawning at 4 users/second against `http://localhost:8080`:

- **Total Requests**: 5,400 requests
- **p50 (Median) Latency**: **82 ms**
- **p95 Latency**: **135 ms** (Max latency threshold is 5,000 ms; achieved 37× faster)
- **Throughput**: **45.0 requests/second**
- **Error Rate**: **0.11%** (6 failed requests out of 5,400, well below 1% limit)
- **Resource Consumption**: CPU: 240m–280m per pod; Memory: 231 MiB per pod (from `kubectl top pods`)

7.2 Cost Estimation (per 1,000 Requests)

Based on the hypothetical pricing in `data/rate_card.json`:

- vCPU cost: INR 3.50 per vCPU-hour; Memory cost: INR 0.45 per GiB-hour
- Limits per replica: 1 vCPU and 1 GiB memory

$$\text{Hourly cost per replica} = (1.0 \times 3.50) + (1.0 \times 0.45) = 3.95 \text{ INR/hour} \tag{4}$$

$$\text{Hourly cost of 2 running replicas} = 2 \times 3.95 = \mathbf{7.90} \text{ INR/hour} \tag{5}$$

$$\text{Requests served per hour} = 45.0 \text{ req/s} \times 3600 \text{ s} = \mathbf{162,000} \text{ requests/hour} \tag{6}$$

$$\text{Cost per 1,000 requests} = \frac{7.90}{162000} \times 1000 = \mathbf{0.04876} \text{ INR} \approx \mathbf{0.049} \text{ INR} \tag{7}$$

7.3 Service Level Objective (SLO) and Error Budget

- **SLO Target**: 99.0% of requests return HTTP 200 within 30 seconds.
- **Window**: One standard 2-minute load-test run (5,400 total requests).
- **Error Budget Calculation**:

$$\text{Total Error Budget} = (1 - 0.99) \times 5400 = \mathbf{54} \text{ requests} \tag{8}$$

- **Error Budget Consumed**: Actual failures observed = 6 requests.

$$\text{Budget Consumed} = \frac{6}{54} \times 100 = \mathbf{11.11}\% \quad \text{(Remaining: 88.89\%)} \tag{9}$$

7.4 System Review Table

Table 6: System Evaluation, Limitations, and Proposed Improvements

Component	**Evidence (Measured)**	**Limitation Found**	**Improvement Proposed**
Data and storage	2,906 clean records; 47 duplicates	Sharded workers require shared filesystem or external S3 coordination	Implement an SQS/RabbitMQ message queue for dynamic chunk leasing
Model quality	99.0% JSON validity; 96.0% full-frame	Pythia-70M struggles with multi-intent or implicit dates	Fine-tune Pythia-160M or incorporate token-level distillation loss
Latency & throughput	p50: 82 ms, p95: 135 ms, 45.0 req/s	Python standard HTTP server uses single-threaded event dispatching	Deploy Uvicorn with Gunicorn ASGI multi-worker processes
Resource utilisation	231.2 MiB peak RSS; 28% CPU utilization	High baseline memory allocation for small 70M model weights	Quantize model weights to 4-bit/8-bit integer formats via ONNX/llama.cpp
Reliability & recovery	4.0s median cold-start; 0.33% recovery loss	Cold-starts incur S3 download network latency on newly spawned pods	Mount shared ReadWriteMany PVC cache or pre-warm container volumes
Security and cost	0.049 INR / 1,000 req; 0 secret leaks	Single cluster-wide S3 credential without IAM granular scoping	Implement AWS IRSA (IAM Roles for Service Accounts) / HashiCorp Vault